Missing integer

introduction

The purpose of this project is to find the smallest positive missing integer from an array of integers.

Examples:

A = {1,3,6,4,1,2}

Output=5; The positive integers 1, 2, 3, and 4 exist in the array. The smallest missing positive integer is 5.

A = {1,2,3}

Output=4; All positive integers from 1 to 3 exist in the array. The next missing positive integer is 4.

A = {-1,-3}

Output=1; There are no positive integers in the array. Therefore, the smallest missing positive integer is 1.

Main class

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        int[] A = {1, 3, 6, 4, 1, 2};
        int[] B = {1, 2, 3};
        int[] C = {-1, -2};
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter array size: ");
        int N = sc.nextInt();
        if (N < 1 || N > 100000) {
            System.out.println("Array size must be between
1 and 100000");
            return;
        }
        int[] d = new int[N];
        System.out.println("Enter array elements:");
        for (int i = 0; i < N; i++) {
```

```
d[i] = sc.nextInt();


            if (d[i] < -1000000 || d[i] > 1000000) {
                System.out.println(
                    "Elements must be between -1000000 and
1000000"
                );
                return;
            }
        }
        System.out.println("User Array:");
        System.out.println(
            "Non Recursive: " +
            MissingIntegerNonRecursive.findMissing(d)
        );
        System.out.println(
            "Recursive: " +
            MissingIntegerRecursive.findMissing(d)
        );
```

```
   System.out.println("\nArray A:");
        System.out.println("Non Recursive: " +
                MissingIntegerNonRecursive.findMissing(A));
        System.out.println("Recursive: " +
                MissingIntegerRecursive.findMissing(A));

System.out.println("\nArray B:");
        System.out.println("Non Recursive: " +
                MissingIntegerNonRecursive.findMissing(B));
        System.out.println("Recursive: " +
                MissingIntegerRecursive.findMissing(B));

        System.out.println("\nArray C:");
        System.out.println("Non Recursive: " +
                MissingIntegerNonRecursive.findMissing(C));
        System.out.println("Recursive: " +
                MissingIntegerRecursive.findMissing(C));
    }
}
```

First Algorithm (Non-Recursive)

Idea

This algorithm uses a boolean array to mark the positive integers that already exist in the array. After marking the existing numbers, the algorithm searches for the first missing positive integer.

Pseudocode

```
1. Let N = length of array A
2. Create boolean array present of size N+1
3. For each element x in A:
            If x > 0 AND x ≤ N:
                  present[x] = true
4. For i from 1 to N:
         If present[i] == false:
                return i
5. Return N + 1
```

Implementation

```
public class MissingIntegerNonRecursive {
                    public static int findMissing(int[] A) {
                                int N = A.length;
                                boolean[] present = new boolean[N + 1];
                                for (int i = 0; i < N; i++) {
                                            if (A[i] > 0 && A[i] <=
N) {

present[A[i]] = true;
                                            }
                                }
                               for (int i = 1; i <= N; i++) {
                                            if (!present[i]) {

return i;
                                            }
                                }
                                return N + 1;
                    }
}
```

Analysis & Complexity (Steps)

Step 1: Get Array Size

```
int N = A.length;
```

The algorithm first stores the size of the array.

Step 2: Create Boolean

```
 Arrayboolean[] present = new boolean[N + 1];
```

A boolean array is created to track which positive integers exist in the array.

Step 3: Traverse the Array

```
 for (int i = 0; i < N; i++)
```

The algorithm loops through all array elements.

Step 4: Check Valid Positive Numbers

```
 if (A[i] > 0 && A[i] <= N)
```

Only positive integers within the range $1 \rightarrow$ N are considered.

Step 5: Mark Existing Numbers

```
present[A[i]] = true;
```

If a number exists, its index in the boolean array becomes true.

Step 6: Find the Missing Integer

```
for (int i = 1; i <= N; i++)
```

The algorithm checks the boolean array to find the first number marked as false.

Step 7: Return the Missing Number

```
 if (!present[i])
return i;
```

The first missing positive integer is returned.

Step 8: Return N + 1

```
return N + 1;
```

If all numbers from $1 \rightarrow N$ exist, the answer will be $N + 1$.

Time Complexity

First Loop $O(N)$

Second Loop$O(N)$

Total Time Complexity: $O(N) + O(N) = O(N)$

Second Algorithm (Recursive)

Idea

This algorithm first sorts the array, then recursively checks the expected smallest positive integer.

Pseudocode:

```
1. Sort array A
2. Call find(A, 0, 1)
     Function find(A, index, expected):
                        If index ≥ length:
                                return expected
                        If A[index] == expected:
                                return find(index+1,
expected+1)
                        Else if A[index] < expected:
                                return find(index+1,
expected)
                        Else:
                                return expected
```

Implementation

```
 import java.util.*;
public class MissingIntegerRecursive {

    public static int findMissing(int[] A) {
        bubbleSort(A);
        return find(A, 0, 1);
    }

    private static void bubbleSort(int[] A) {
        int N = A.length;
        for (int i = 0; i < N - 1; i++) {
            for (int j = 0; j < N - i - 1; j++) {
                if (A[j] > A[j + 1]) {
                    int temp = A[j];
                    A[j] = A[j + 1];
                    A[j + 1] = temp;
                }
            }
        }
    }
```

```
private static int find(int[] A, int index, int
expected) {

        if (index >= A.length) {
            return expected;
        }

        if (A[index] == expected) {
            return find(A, index + 1, expected +
1);
        } else if (A[index] < expected) {
            return find(A, index + 1, expected);
        } else {
            return expected;
        }
    }
}
```

Analysis & Complexity (Steps)

Step 1: Sort the Array

bubbleSort(A);

the array is sorted using Bubble Sort.

Step 2: Bubble Sort Traversal

for (int i = 0; i < N - 1; i++)

for (int j = 0; j < N - i - 1; j++)

Nested loops compare adjacent elements.

Step 3: Swap Elements if (A[j] > A[j + 1])

if elements are in the wrong order, they are swapped.

Step 4: Start Recursive Search

return find(A, 0, 1);

The recursive function starts from:

index = 0

expected = 1

Step 5: Base Case

if (index >= A.length)

If the algorithm reaches the end of the array, it returns the expected value.

Step 6: Found Expected Number

```
if (A[index] == expected)
```

If the current element equals the expected number:

Move to the next element.

Increment expected value.

Step 7: Ignore Smaller Numbers

```
else if (A[index] < expected)
```

If the current number is smaller than expected, it is ignored.

Step 8: Missing Number Found else

```
return expected;
```

If the current number is larger than expected, the expected number is missing.

Bubble Sort Complexity:

The outer loop runs N times. The inner loop also runs approximately N times. Total operations: N × N Therefore: O(N²)

Recursive Search Complexity:

The recursive function visits each element once. Therefore: O(N) Overall Complexity: O(N²) + O(N) The dominant complexity is: O(N²)

Comparison Between Algorithms

Feature	Non-recursive	recursive
technique	Boolean hashing	Bubble sort + recursion
Time complexity	O(N)	O(N²)
performance	Faster	slower
simplicity	medium	Easy to understand
Practical use	preferred	educational

Conclusion

The project demonstrates two different approaches for solving the Missing Integer problem. The Non-Recursive algorithm is more efficient because it works in linear time $O(N)$. It is considered the best practical solution. The Recursive algorithm is slower because it uses Bubble Sort with complexity $O(N^2)$, but it is useful for educational purposes because it demonstrates recursion and sorting clearly. The project also highlights the difference between iterative and recursive problem-solving techniques.

Github link:

https://github.com/AhmedHossam06/Missing_Integer